OFFICIAL COURSE PROPOSAL

Problem Solving & Algorithmic Thinking

From Zero Foundations to Intermediate Competitive Mastery

TARGET AUDIENCE
Level 0 Beginners to Coders

DELIVERY MODE
Hands-on Interactive Lab

GOAL
Technical Interview & Contest Ready

COURSE INTRODUCTION

Bridging the Gap Between Code Syntax and Logic

The Core Challenge

Students learn syntax easily, yet fail when facing unseen algorithmic problems during interviews and contests.

The Structured Solution

A step-by-step framework converting raw problem statements into efficient code via systematic mental patterns.

Measurable Outcome

Students solve Codeforces Div 2 (A/B) and LeetCode Medium problems independently with time complexity awareness.

LEARNING OUTCOMES

Core Student Objectives Upon Completion

Algorithmic Thinking Pattern

Break down complex real-world problems into mathematical models and structured pseudocode step-by-step.

Time & Space Efficiency

Calculate Big-O time complexity accurately and optimize algorithms from $O(N^2)$ down to $O(N \log N)$ or $O(1)$.

Core Data Structure Mastery

Master Arrays, Vectors, Strings, Maps, Sets, Stacks, Queues, and Graph traversals in practice.

Competitive Coding Readiness

Navigate Online Judges (Codeforces, LeetCode, VJudge) and pass automated testing constraints confidently.

ENTRY REQUIREMENTS

Who Is This Course Designed For?

Target Audience

Ideal Student Profiles

- CS Freshmen seeking foundational strength
- Self-taught coders lacking computer science fundamentals
- Students targeting FAANG & tech interview prep
- Aspiring ICPC and competitive programming participants

Prerequisites

Zero High-Level Requirements

- Basic Computer Literacy & typing ability
- Basic High School Algebra (Variables & Functions)
- Zero prior coding experience strictly required
- High commitment to daily problem solving

CURRICULUM STRUCTURE

4-Stage Progressive Learning Roadmap

STAGE 01

Level 0: Basics

I/O, Loops, Conditionals, Arrays, and Syntax Control.

STAGE 02

Level 1: Foundations

Complexity, Strings, Vectors, Maps/Sets, and Frequency Arrays.

STAGE 03

Level 2: Techniques

Two Pointers, Prefix Sum, Binary Search, Greedy Choices.

STAGE 04

Level 3: Intermediate

Recursion, Number Theory, Basic Graphs (BFS/DFS), and DP Intro.

MODULE 1 SYLLABUS

Level 0: Syntax, I/O & Basic Building Blocks

Key Topics Covered

- Environment setup (VS Code, C++ STL Compiler)
- Fast Input/Output standard operations
- Conditional Logic & Nested Branching
- Iteration Mechanics (For, While, Do-While loops)
- 1D & 2D Static Array Manipulations

Practical Milestones

- Solving 30+ Level-0 Practice Problems on VJudge
- Avoiding common TLE & Syntax runtime errors
- Pattern Printing & Math Series Summation
- Building logic without relying on built-in functions

MODULE 2 SYLLABUS

Level 1: Complexity Analysis & STL Data Structures

Key Topics Covered

- Big-O, Time Limits (10^8 ops/sec rule), Space Bounds
- String Manipulation & Fast Hashing Concepts
- Dynamic Arrays (`std::vector`) & Iterators
- Frequency Arrays & Counting Techniques
- Associative Containers (`std::map`, `std::set`)

Practical Milestones

- Analyzing problem constraints before writing code
- Solving Anagram & Character Counting problems
- Finding unique elements and range queries in $O(\log N)$
- Contest 1: Standard Template Library Sprint

MODULE 3 SYLLABUS

Level 2: Essential Algorithmic Paradigm Paradigms

Core Algorithms

- Two Pointers Technique (Sliding Window)
- Prefix Sums & Partial Sum Queries
- Binary Search on Answer & Fractional Bisection
- Sorting Mechanics & Custom Comparators
- Greedy Strategies & Constructive Algorithms

Problem Solving Target

- Transforming $O(N^2)$ brute-force to $O(N)$ sliding window
- $O(1)$ Range Sum Queries execution
- Binary searching monotonic functions
- Tackling Codeforces Div 2 (A & B) problems

MODULE 4 SYLLABUS

Level 3: Intermediate Math, Graphs & DP

Advanced Topics

- Recursion Trees & Backtracking Foundations
- Basic Number Theory (Sieve, GCD, Modular Math)
- Graph Representation (Adjacency Matrix & List)
- Breadth-First Search (BFS) & Depth-First Search (DFS)
- Dynamic Programming (Memoization & 0/1 Knapsack)

Intermediate Mastery

- Finding shortest paths on unweighted graphs
- Generating prime factors in $O(\log N)$ time
- Solving classic DP problems independently
- Solving LeetCode Medium / Codeforces C problems

PEDAGOGY & DELIVERY

How the Course Is Taught: The Active Learning System

1. Live Modeling

Live coding with purposeful mistakes to teach debugging, edge-case identification, and dry-running on whiteboards.

2. In-Class Drills

15–minute solo problem-solving sprints following concept introductions for immediate retention.

3. Group Editorial

Students present different approaches (brute force vs optimized) to foster peer learning and discussion.

EVALUATION METRIC

Multi-Tier Assessment Framework

40%

Weekly Sheet Submissions

Curated problem sheets (10-15 problems weekly) on automated online judges.

30%

Live Contests

4 Timed competitive contests simulating real interview pressure on VJudge.

30%

Final Capstone & Interview

1-on-1 mock technical interview evaluating algorithmic explanation skills.

READY FOR LAUNCH

Let's Build World-Class Problem Solvers

Complete course materials, automated judges, and problem sheets ready for immediate integration into your academy curriculum.

Accept Course Proposal